SRM UNIVERSITY AP

CSE-211 FULL STACK DEVELOPMENT

SRM UNIVERSITY, AP

X

SMART-BRIDGE

PROJECT REPORT

Rentie - Cloth Rental Platform

(https://github.com/sxvyx/rentie-cloth-rental)

Submitted by:-

NAME	REGISTRATION NUMBER
Saurav Shankar	AP24110010128
Abhinav Kumar Singh	AP24110010746
Aashir Aftab	AP24110010400
Ayush Shukla	AP24110010701

Degree : B.Tech 2nd Year

Section : CSE – C

INTRODUCTION

Rentie is a comprehensive web-based platform designed to revolutionize the way people access fashion for special occasions. Instead of purchasing expensive outfits that may only be worn once, users can browse through a wide variety of clothing items, select outfits based on categories such as wedding, party, or casual wear, check availability for specific dates, and rent items for a defined period all through an intuitive digital interface.

This system provides an easy and efficient way for customers to explore clothing options, make bookings online, and manage their rental history, while simultaneously offering administrators powerful tools to manage clothing inventory, track rentals and returns, and monitor user activity. Built on the MERN (MongoDB, Express, React, Node.js) stack, the application ensures scalability, security, and seamless user experience.

Rentie addresses the growing demand for sustainable fashion solutions and cost-effective access to designer wear, making high-quality clothing accessible to everyone while helping businesses efficiently manage their rental operations digitally.

SCENARIO BASED CASE STUDY

Meet Priya, a young professional with a demanding career who values both style and smart spending. Priya has been invited to her best friend's wedding next month a grand celebration where she wants to look her absolute best. However, she's hesitant to spend a fortune on a designer lehenga that she'll likely wear only once.

One evening, while browsing online for alternatives, Priya discovers the Rentie, a comprehensive web application designed to streamline clothing rental operations. Intrigued by its features, Priya decides to explore how the platform can help her find the perfect outfit without breaking the bank.

User Registration and Authentication:

Priya registers an account on Rentie, providing her personal details and preferences. She logs in securely using her credentials, ensuring that her data remains private and secure.

Clothing Catalog Browsing:

Priya explores the extensive clothing catalog, which includes detailed information about each outfit—designer name, size options, rental price per day, and high-quality images. She can filter by category (wedding, party, casual), price range, and size to find exactly what she's looking for.

Availability Check and Booking:

After finding the perfect designer lehenga, Priya checks its availability for her desired rental dates. The app shows a calendar view with available and booked dates. She selects her rental period (pickup and return dates), reviews the total cost, and confirms her booking.

After selecting her rental period, Priya proceeds to a secure checkout powered by Razorpay, where she pays the rental fee and receives an instant digital invoice.

Order Management:

Priya receives an instant booking confirmation with all the details—order ID, rental period, total price, and pickup instructions. She can track her order status and view her booking history anytime through her profile dashboard.

Return Process:

After the wedding, Priya returns the outfit on time. The system automatically updates the inventory availability, and she receives a confirmation of successful return.

Admin Operations:

Meanwhile, the admin manages the platform efficiently:

- Adds new clothing items to the inventory with descriptions, images, and pricing
- Monitors payment success rates and transaction history through the Razorpay dashboard.
- Tracks all bookings and monitors rental status (booked, in-use, returned)
- Updates availability automatically after returns
- Views user activity and manages customer accounts
- Generates reports on rental trends and revenue

Priya's Experience

Thanks to Rentie, Priya was able to wear a stunning designer lehenga to her friend's wedding at a fraction of the purchase cost. She felt confident and looked amazing without the guilt of an expensive one-time purchase. The seamless booking process, transparent pricing, and easy return made the entire experience stress-free. Rentie has become her go-to solution for special occasions, helping her stay fashionable while making sustainable and economical choices.

SYSTEM REQUIREMENTS

Software Requirements

To ensure smooth development, deployment, and usage of Rentie, certain system prerequisites must be met. These requirements are categorized into software, setup, and hardware specifications, all of which contribute to building a robust and scalable rental platform.

- **Operating System:** Windows 10/11, macOS, or Linux Supports cross-platform development and testing.
- **Node.js (v18 or above):** Provides the runtime environment for building and managing frontend logic and powers the server-side logic and handles API routing.
- **npm (v8 or above):** A package manager required to install dependencies for React and related libraries.
- **React.js & Vite:** A powerful frontend combination for building fast, reactive, and optimized user interfaces.
- **Browser:** Google Chrome / Firefox (latest version) — For rendering and testing the UI in real-time.
- **Express.js:** A lightweight web framework for building RESTful APIs.
- **MongoDB:** A NoSQL database used to store structured and unstructured data related to users, clothes, and bookings.
- **Razorpay SDK & API:** Payment gateway integration used to handle secure transactions and real-time order verification.
- **Postman:** Tool for testing APIs during development.
- **Visual Studio Code:** Preferred code editor with built-in Git and terminal support.

Hardware Requirements

Describes the minimum and recommended specifications needed to support the development and usage of the application.

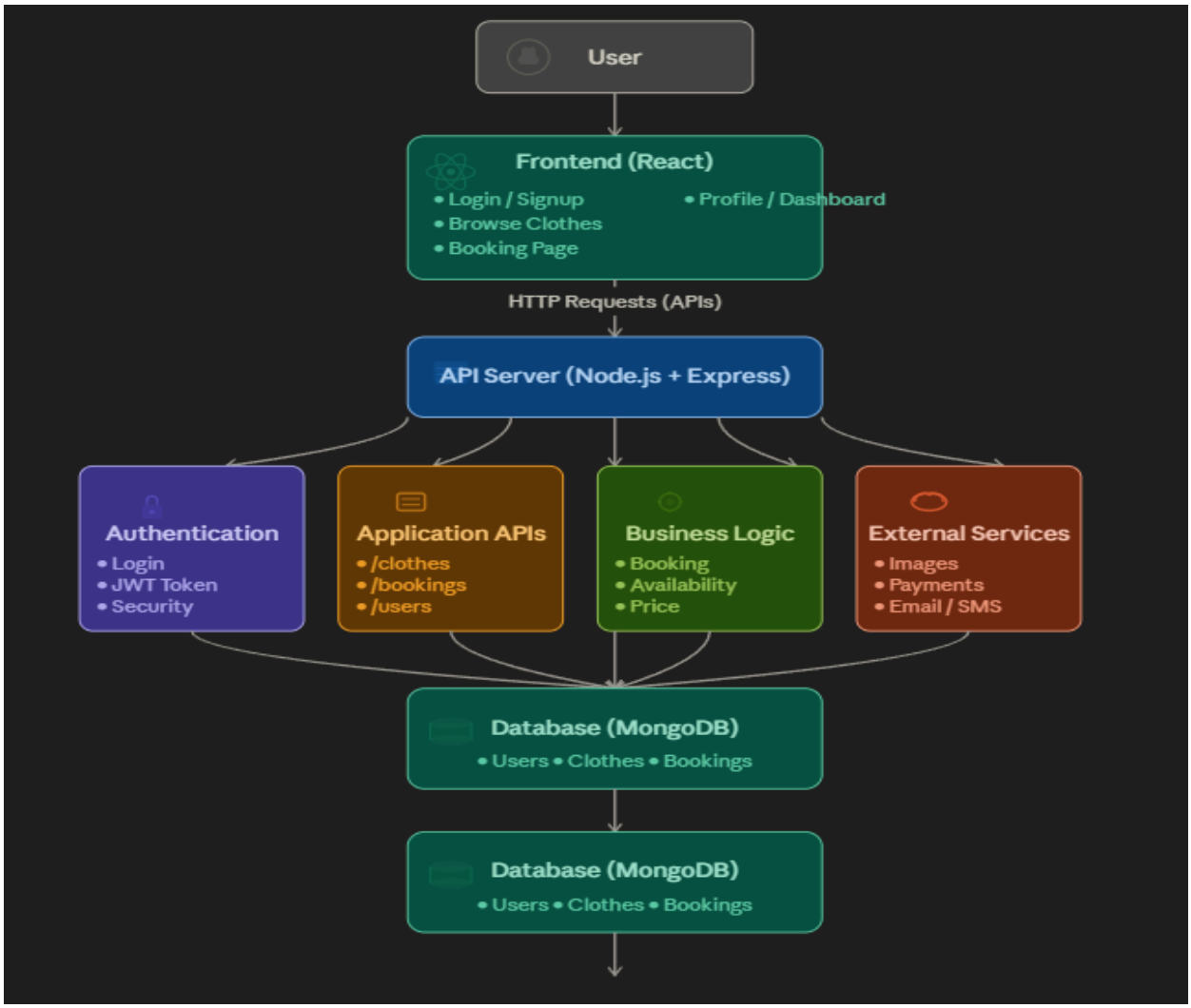
- **Processor:** Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or better — Ensures fast compilation and multitasking during development.
- **RAM:** Minimum 8 GB (16 GB recommended) — For handling development servers, IDEs, and browser testing simultaneously.
- **Storage:** At least 5 GB free space — Required for node_modules, MongoDB data storage, and high-resolution product imagery.
- **Display:** 1920x1080 (Full HD) — Recommended for the best coding experience and verifying high-fidelity UI designs.

PROJECT ARCHITECTURE

Rentie follows a modern 3-tier architecture pattern with additional service layers for enhanced functionality. This architecture ensures separation of concerns, scalability, and maintainability.

High-Level Architecture (3-Tier + Extended)

Frontend (React) → API Layer (REST APIs - Express) → Backend (Node.js Business Logic) → Database (MongoDB)



Detailed Architecture Layers

1. User & Frontend (React)

- The user interacts with the React app
- Performs actions like: Login/Signup, Browse clothes, Book outfits, View profile/dashboard

- Frontend sends requests to backend using REST APIs (HTTP)

2. API Gateway (Node.js + Express)

- This is the entry point of the backend
- It receives all requests from frontend
- Routes requests to correct modules: Authentication, Clothes, Bookings, Users

3. Authentication Module

- Handles: User signup/login, JWT token generation, Token verification, Password hashing
- Ensures security and protected access

4. Application APIs

- /api/product → Endpoints for managing the clothing inventory, including addition, deletion, and listing of items.
- /bookings → Rental booking system
- /users → User profile management

5. Business Logic Layer (Core Brain)

The most important part of the system. Handles:

- Booking management
- Availability check (date overlap prevention)
- Price calculation based on rental duration
- Inventory updates after booking and return

6. External Services

- Image upload → Cloudinary / AWS S3
- Payments → Razorpay Gateway (Integrated for handling secure transactions, order creation, and payment verification).
- Emails → Notification service for booking confirmations

7. Data Access Layer

- Uses Mongoose models to interact with database
- Handles: Queries, Data validation, Relationships

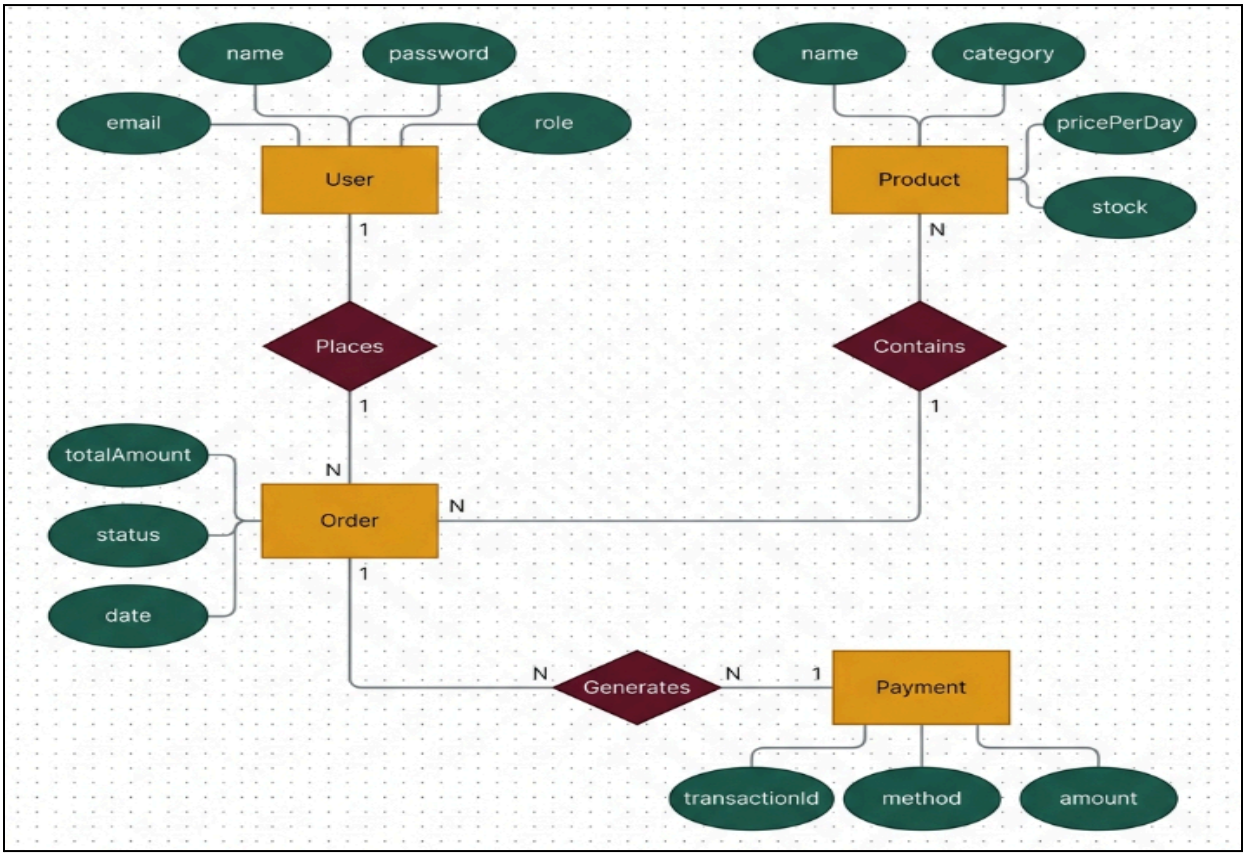
8. Database (MongoDB)

Stores all data:

- Users (authentication and profile)
- Products (Stores detailed inventory information: name, category, price, sizes, and stock availability).
- Bookings (rental records)

ER DIAGRAM

The Entity-Relationship (ER) diagram for Rentie represents the various entities and their relationships within the rental management system. Here is a description of the key components of the ER diagram:



1. Entities

- **User:** Represents users who rent clothing items, with attributes such as user ID, name, email, password (hashed), and role (user/admin).
- **Product:** Represents clothing items available for rent, with attributes such as cloth ID, title, description, category, size, price per day, availability status, and image URL.
- **Order:** Represents rental bookings made by users, with attributes such as booking ID, user ID, cloth ID, start date, end date, total price, and status (booked/returned).
- **Payment:** Represents the transaction details, including Razorpay Payment ID, amount, and status (Success/Pending).

2. Relationships

- **User-Booking:** One-to-Many relationship. One user can have multiple bookings, but each booking belongs to one user.
- **Clothes-Booking:** One-to-Many relationship. One clothing item can be booked multiple times (different dates), but each booking references one specific clothing item.

- **Order-Payment:** One-to-One relationship. Every successful order is linked to a unique transaction record in the database.

3. Attributes

Each entity has its own set of attributes that describe its properties. For example, the Clothes entity has attributes such as title, description, category, size, price per day, availability, and image URL.

4. Primary Keys

Each entity has a primary key that uniquely identifies each record in the entity. For example, the User entity's primary key is the user ID (MongoDB ObjectId).

5. Foreign Keys

Foreign keys are used to establish relationships between entities. For example, the Booking entity has foreign keys userId and clothId to link bookings to users and clothing items respectively.

Overall, the ER diagram for Rentie provides a visual representation of the database schema, showing how different entities are related and how data flows between them in the rental management system.

KEY FEATURES

User Features

- **User Authentication:** Secure login and registration with JWT-based authentication
- **Browse Clothing Catalog:** Explore wide variety of clothing items with detailed descriptions and images
- **Filter/Search Functionality:** Filter by category (wedding, party, casual), price range, size, and availability
- **Availability Check:** Check real-time availability for specific dates before booking
- **Rental Booking System:** Book clothing items for specific rental periods with automatic price calculation
- **Booking History:** View past and current bookings with status tracking
- **Profile Dashboard:** Manage personal information and view rental history
- **Return Tracking:** Track return dates and receive notifications

Admin Features

- **Admin Authentication:** Secure admin login with role-based access control
- **Inventory Management:** Add, edit, and delete clothing items from the catalog
- **Booking Management:** View all bookings, track rental status, and manage returns
- **User Management:** View and manage registered users
- **Availability Updates:** Automatically update item availability after bookings and returns
- **Admin Dashboard:** Comprehensive overview of users, inventory, and bookings with analytics

Optional Advanced Features

- Online Payment Integration (Razorpay/Stripe)
- Real-time Availability Calendar
- Ratings & Reviews for clothing items
- Email/SMS Notifications for booking confirmations and reminders
- Admin Analytics Dashboard with charts and reports
- Wishlist Feature for users to save favorite items
- AI-based Recommendations based on user preferences and history

ROLES AND RESPONSIBILITIES

User Flow

User Journey:

1. **Registration & Login:** User creates an account or logs in to access the platform
2. **Browse Clothes:** User explores available clothing categories and items
3. **Filter by Category/Price/Date:** User applies filters to narrow down search results
4. **Select Outfit:** User selects a desired clothing item to view details
5. **Choose Rental Dates:** User selects pickup and return dates for the rental period
6. **Check Availability:** System verifies if the item is available for the selected dates
7. **Confirm Booking:** User confirms the booking and receives a confirmation notification
8. **Payment (Optional):** User completes payment if payment gateway is integrated
9. **Receive Confirmation:** User receives booking details via email/notification
10. **Return Item:** User returns the item after the rental period ends

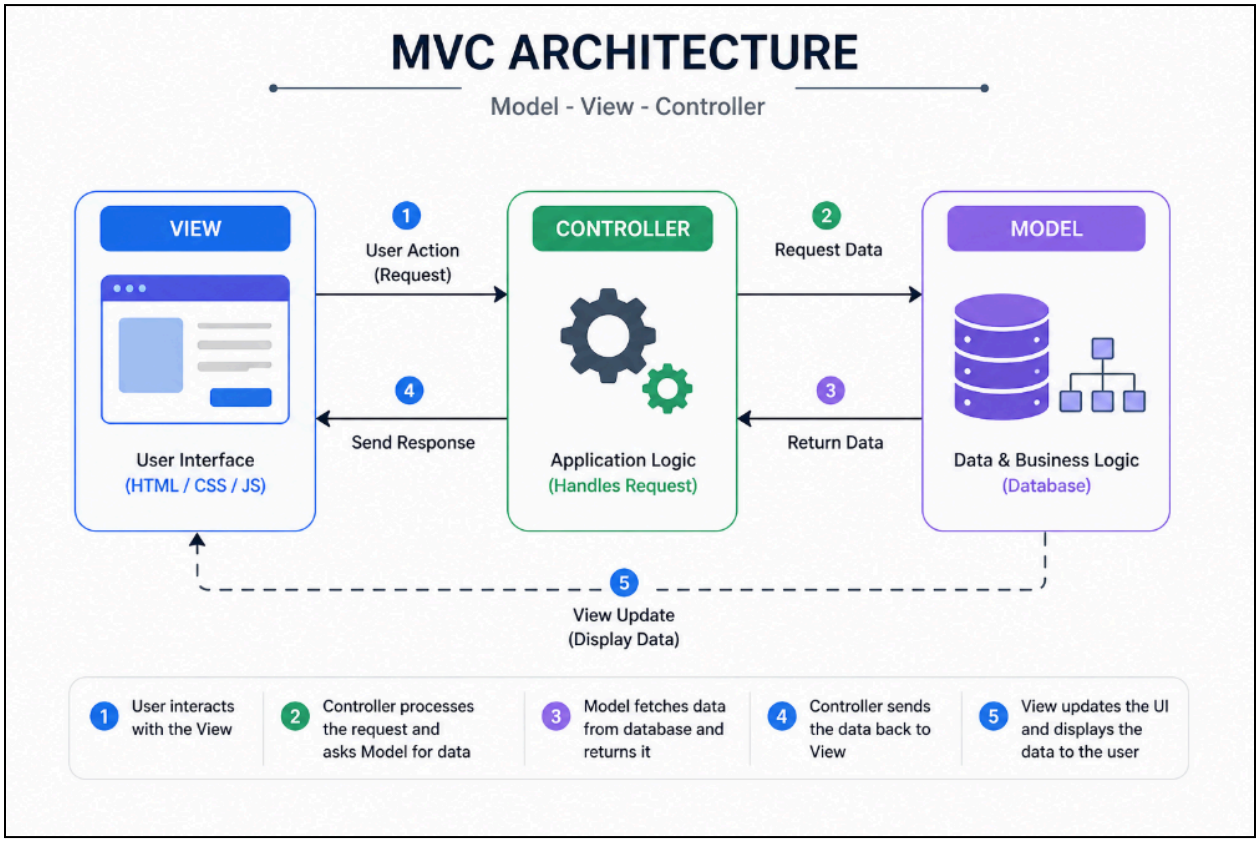
Admin Flow

Admin Operations:

11. **Login:** Admin securely logs into the admin panel
12. **Add New Clothing Item:** Admin adds new clothing items to the inventory with details (title, description, category, size, price, image)
13. **Update Availability:** Admin manually or automatically updates item availability based on bookings and returns
14. **View Bookings:** Admin monitors all current and past bookings
15. **Manage Users:** Admin views registered users and their activity
16. **Mark Item as Returned:** Admin confirms item returns and updates system status
17. **Generate Reports:** Admin generates analytics and reports on rental trends and revenue

MVC PATTERN

Rentie follows the Model-View-Controller (MVC) architectural pattern, a software design approach that separates an application into three interconnected layers. This separation allows for modularity, easier maintenance, and scalability.



Model Layer (Data Layer)

The Model layer is responsible for handling all data-related logic. This includes the definition of data schemas and the operations performed on the database using those schemas. The models are implemented using Mongoose, which provides a schema-based solution to model application data for MongoDB.

Key Models:

- **User Model:** Defines user schema with fields like name, email, password, and role
- **Booking Model:** Defines clothing item schema with fields like title, description, category, size, price, availability, and imageUrl
- **Payment Model:** Stores transaction IDs and payment statuses retrieved from the Razorpay API.
- **Booking Model:** Defines booking schema with fields like userId, clothId, startDate, endDate, totalPrice, and status

Controller Layer

The Controller layer acts as an intermediary between the view (routes) and the model. It receives incoming requests, processes the input (which may include validation or transformation), calls the appropriate methods from the model, and then returns a response to the client.

Key Controllers:

- **Auth Controller:** Handles user registration, login, token generation, and authentication
- **Clothes Controller:** Manages clothing inventory operations (CRUD)
- **Booking Controller:** Handles booking creation, availability checks, and booking management
- **User Controller:** Manages user profile operations

View Layer (Routing Layer)

In the context of a backend REST API, the View is implemented as the routing layer, where various endpoints are defined. These endpoints determine how the backend responds to different HTTP requests (GET, POST, PUT, DELETE) and are responsible for invoking the appropriate controller functions.

Key Routes:

- **/api/auth:** Authentication endpoints (register, login)
- **/api/clothes:** Clothing inventory endpoints (get all, get by ID, create, update, delete)
- **/api/bookings:** Booking management endpoints (create booking, get bookings, update status)
- **/api/users:** User management endpoints

Advantages of Using MVC in This Project

- **Separation of Concerns:** Each layer has a clearly defined responsibility, improving readability and maintainability
- **Scalability:** New features can be added easily by creating new routes, controllers, and models
- **Reusability:** Logic in controllers and models can be reused across multiple parts of the application
- **Testing:** Each layer can be tested independently, especially the controllers and models
- **Collaboration-Friendly:** Multiple developers can work simultaneously on different layers without conflict

PROJECT SETUP AND CONFIGURATION

GitHub Repository Access (Cloning)

If you are setting up the project from an existing repository, follow these steps:
- Clone the project: git clone https://github.com/sxvyx/rentie-cloth-rental.git
- Navigate to the project folder:
  cd rentie-cloth-rental
- Install Backend Dependencies:
  cd backend && npm install
- Install Frontend Dependencies:
  cd ../frontend && npm install
- Configure Environment Variables:
  Create a .env file in the backend folder and add your MONGO_URI, JWT_SECRET, and RAZORPAY_KEY_ID.

Creating Project Folder

1.Create a root folder: rentie-project
2.Inside it, create two directories: frontend and backend.
3.Open the root folder in Visual Studio Code.

Client Setup (Installing React App)

Step 1: Open the client folder in terminal

npm create vite@latest . -- --template react

Step 2: Select React framework from the given options

Step 3: Select JavaScript variant

Step 4: Navigate to the client folder

cd client

Step 5: Install all packages

npm install

Step 6: Start the React development server

npm run dev

Backend(Node.js+Express)

1. Navigate to the backend directory.
2. Initialize Node: npm init -y
3. Create the main entry file: index.js (or server.js).

4. Create the folder architecture:
   models/, controllers/, routes/, middleware/, config/
5. Install backend dependencies: npm install express mongoose jsonwebtoken bcryptjs cors dotenv razorpay
6. Install development tools: npm install --save-dev nodemon

DATABASE DEVELOPMENT

1. Configure MongoDB

Install Mongoose:

npm install mongoose

Create Database Connection

Create a file config/db.js to establish MongoDB connection:

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
    console.log('MongoDB Connected');
  } catch (error) {
    console.error(error);
    process.exit(1);
  }
};

module.exports = connectDB;
```

Create Schemas

Configure the Mongoose schemas for MongoDB database to store data in a structured pattern. Use the data from the ER diagrams to create the schemas.

1) User Schema (models/User.js)

```
const mongoose = require('mongoose');
const UserSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['user', 'admin'], default:
'user' },
}, { timestamps: true });
module.exports = mongoose.model('User', UserSchema);
```

2) Product Schema (models/Product.js)

```
const mongoose = require('mongoose');
const ProductSchema = new mongoose.Schema({
  name: { type: String, required: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  price: { type: Number, required: true }, // Rental price per day
  image: { type: String, required: true }, // Cloudinary URL
  available: { type: Boolean, default: true },
}, { timestamps: true });
module.exports = mongoose.model('Product', ProductSchema);
```

3) Order Schema (models/Order.js)

```
const mongoose = require('mongoose');
const OrderSchema = new mongoose.Schema({
  userId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  products: [{ type: mongoose.Schema.Types.ObjectId, ref: 'Product' }],
  amount: { type: Number, required: true },
  status: { type: String, default: "Pending" }, // Pending, Paid, Completed
  date: { type: Date, default: Date.now }
}, { timestamps: true });
module.exports = mongoose.model('Order', OrderSchema);
```

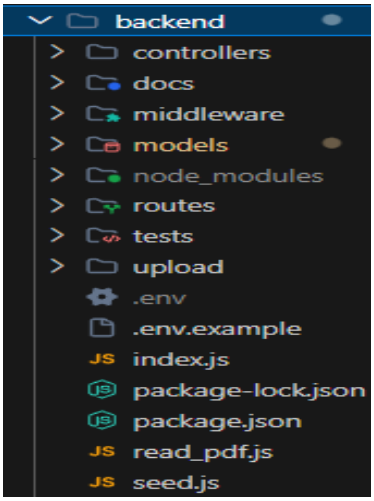
4) Payment Schema (models/Payment.js)

```
const mongoose = require('mongoose');
const PaymentSchema = new mongoose.Schema({
  orderId: { type: String, required: true },
  razorpay_payment_id: { type: String, required: true },
  razorpay_order_id: { type: String, required: true },
  status: { type: String, required: true },
}, { timestamps: true });
module.exports = mongoose.model('Payment', PaymentSchema);
```

BACKEND DEVELOPMENT

(Frontend Deployment: https://rentie-backend.onrender.com)

Backend Structure



Controllers

- **authController.js:** Manages user registration, secure login, and JWT generation.
- **productController.js** Handles CRUD operations for the clothing inventory (Listing, Adding, and Deleting).
- **orderController.js:** Manages the rental lifecycle, from order placement to return status.
- **paymentController.js:** Integrates the Razorpay API to handle payments and verify digital signatures.

Routes

- **authRoutes.js:** Endpoints for user authentication (/api/auth/register, /api/auth/login).
- **productRoutes.js:** Endpoints for the clothing catalog (/api/products).
- **orderRoutes.js:** Endpoints for booking management (/api/orders).
- **paymentRoutes.js:** Endpoints for transaction processing (/api/payment/create, /api/payment/verify).
- **userRoutes.js:** Endpoints for managing user profiles and admin lists.

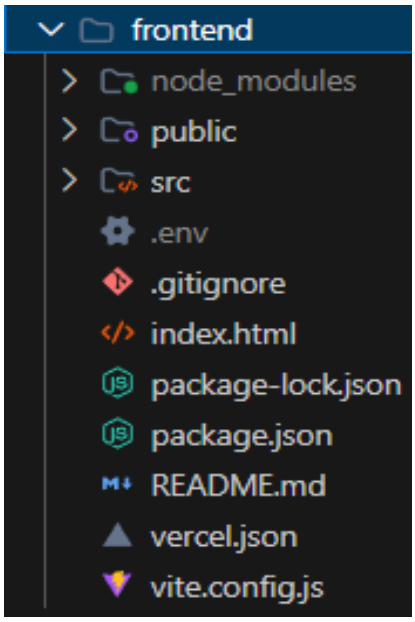
Middleware

- authMiddleware.js: A security layer that verifies the JWT (JSON Web Token) before allowing access to protected routes (like the Admin Dashboard or User Profile).
- uploadMiddleware.js: (If using Multer/Cloudinary) Handles image parsing and storage during product addition.

FRONTEND DEVELOPMENT

(Frontend Deployment: https://rentie-frontend.vercel.app/)

- Structure



- User Pages

- **Home.jsx:** Landing page with featured clothing items
- **Login.jsx:** User login page
- **Register.jsx:** User registration page
- **ClothesCatalog.jsx:** Browse all available clothing items with filters
- **ClothDetails.jsx:** Detailed view of a clothing item with booking option
- **MyBookings.jsx:** User's booking history and active rentals

- Admin Pages

- **AdminDashboard.jsx:** Admin homepage with analytics and overview
- **AddClothes.jsx:** Form to add new clothing items to inventory
- **ManageBookings.jsx:** View and manage all bookings
- **ManageUsers.jsx:** View and manage registered users

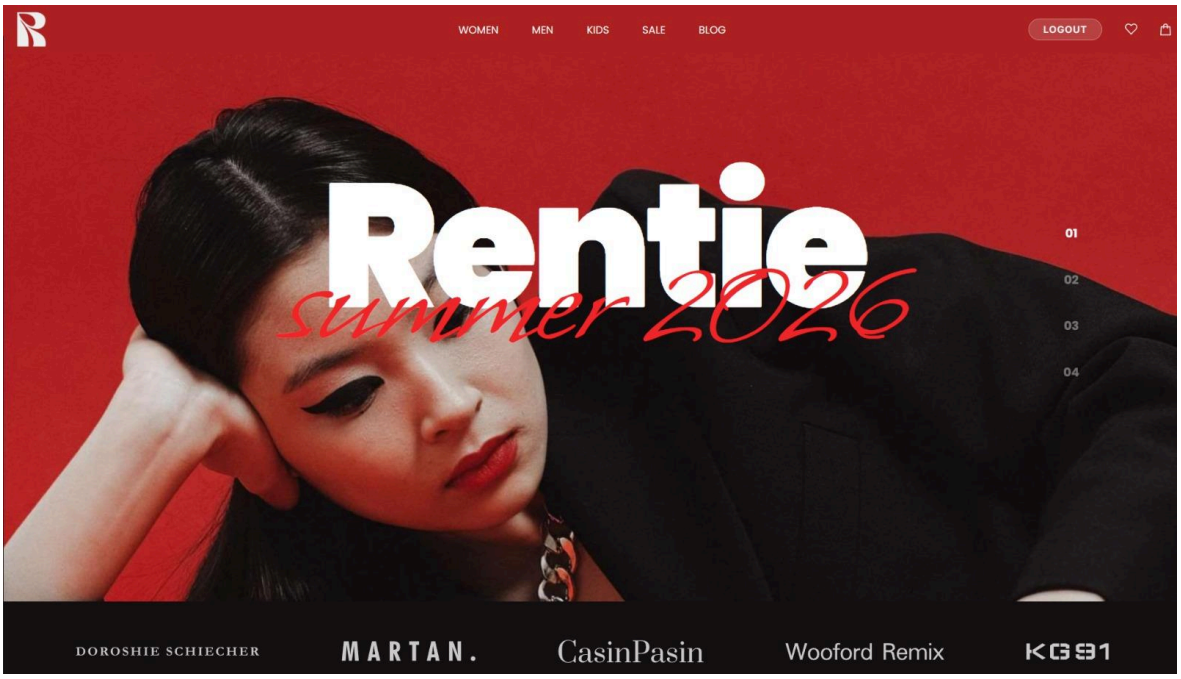
- Reusable Components

- **Navbar.jsx:** Navigation bar component
- **Footer.jsx:** Footer component
- **ClothCard.jsx:** Card component to display clothing item preview
- **BookingForm.jsx:** Form component for creating bookings

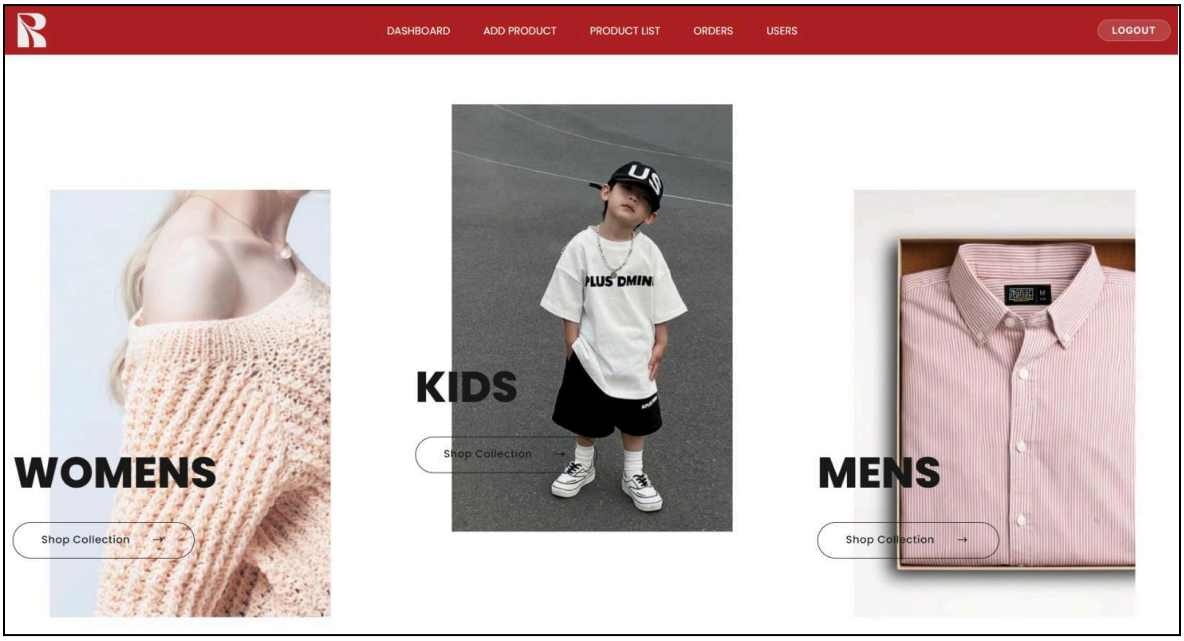
OUTPUT SCREENSHOTS

This section contains screenshots of the various pages and features of Rentie. Insert your actual screenshots in place of the placeholders below.

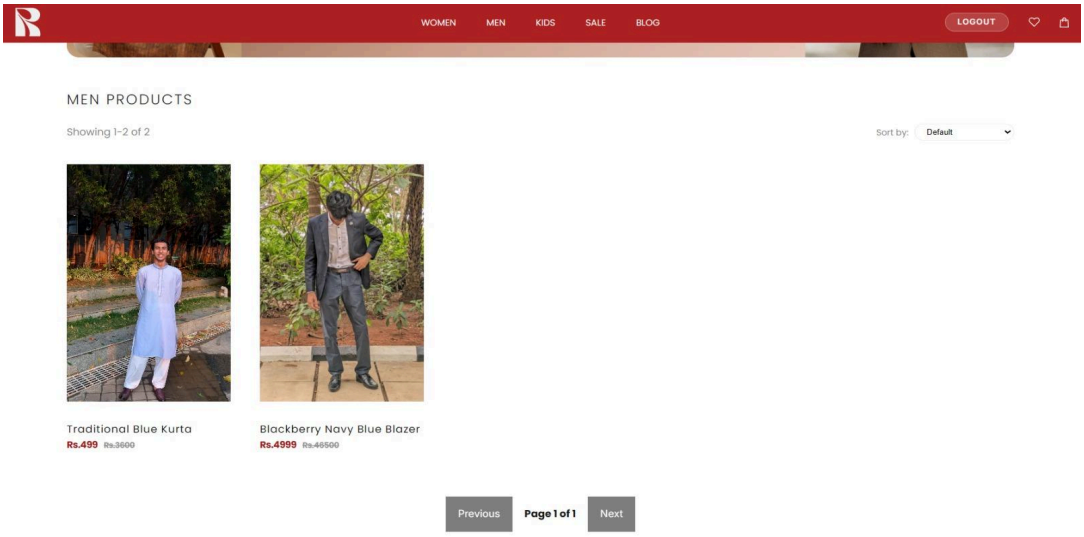
Landing Page



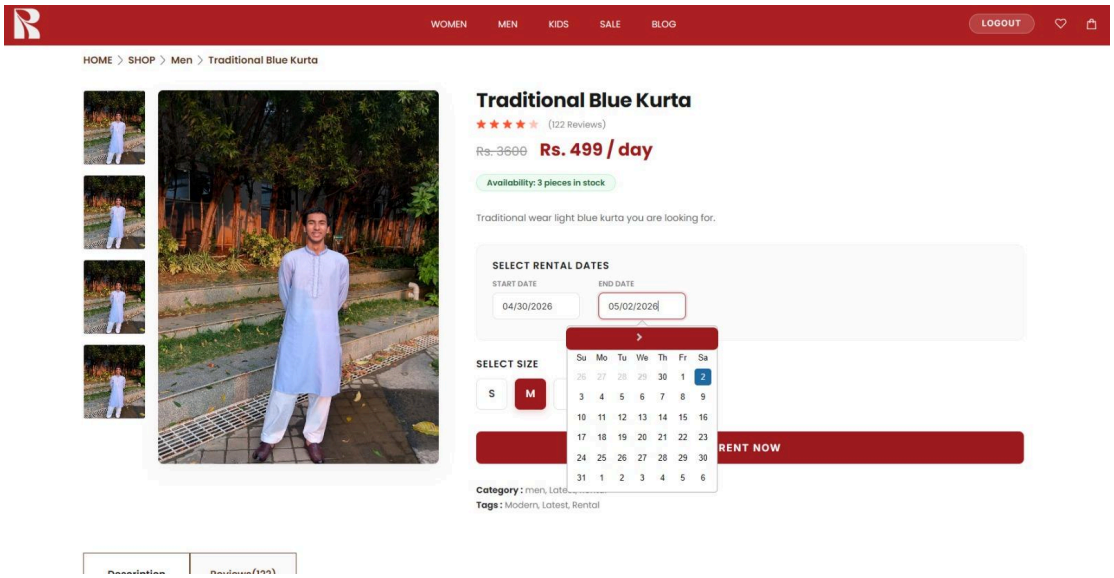
User Veiw Page



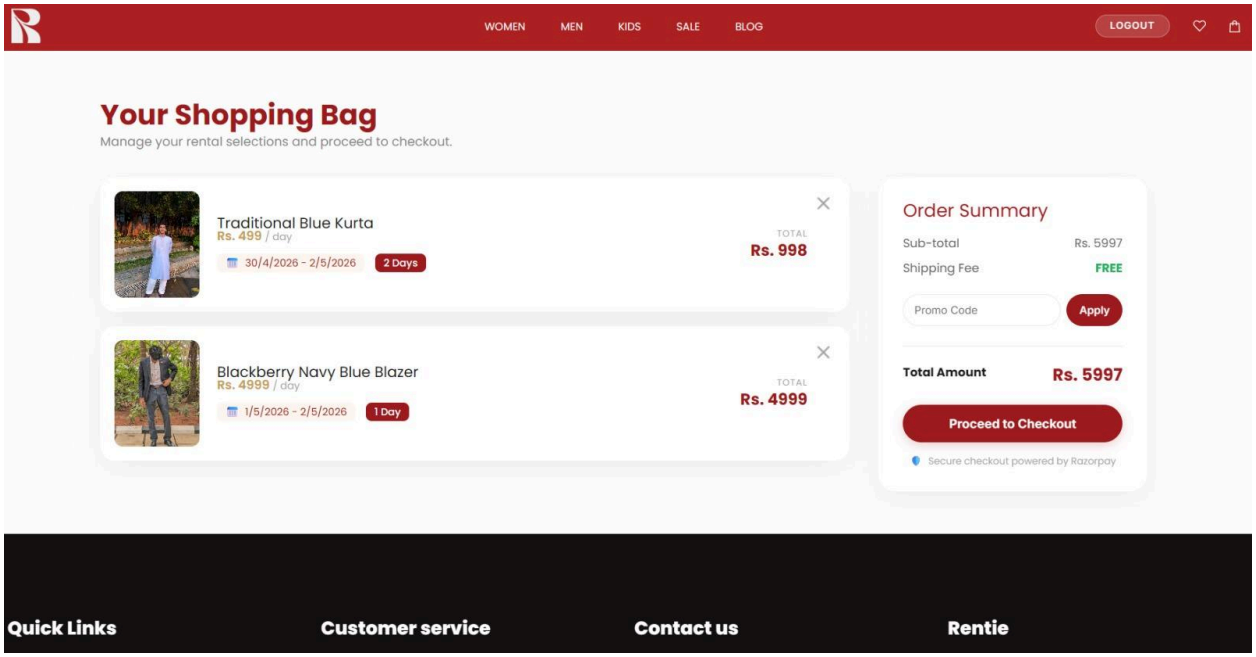
Clothes Catalog/Browse Page



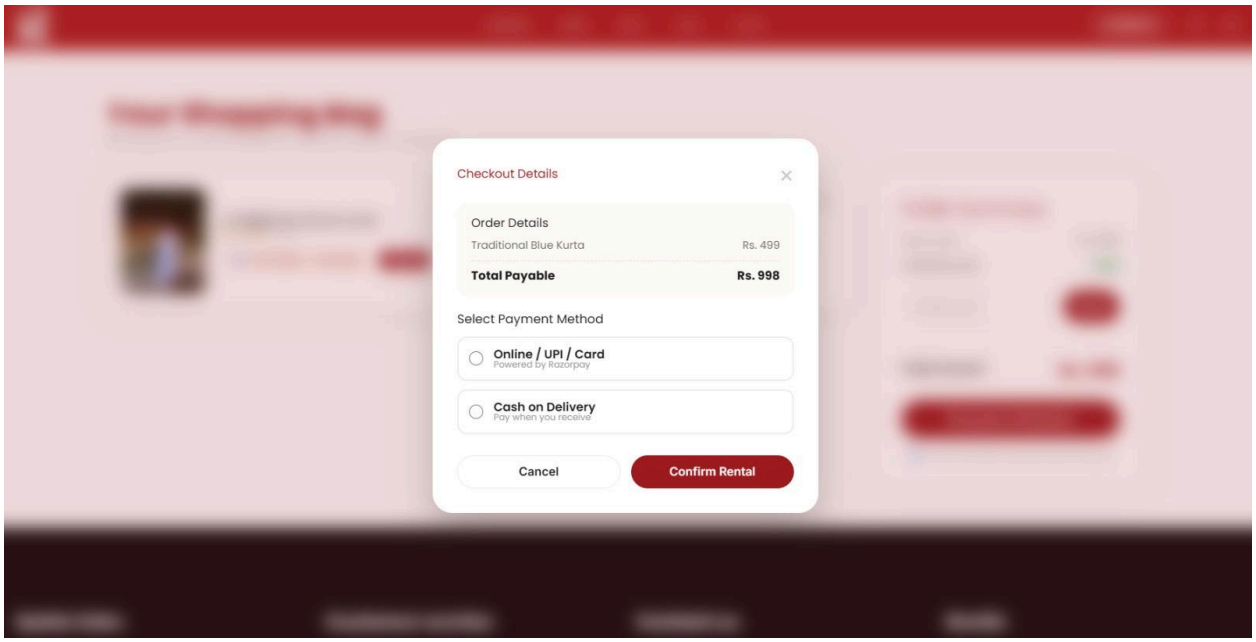
Booking Form



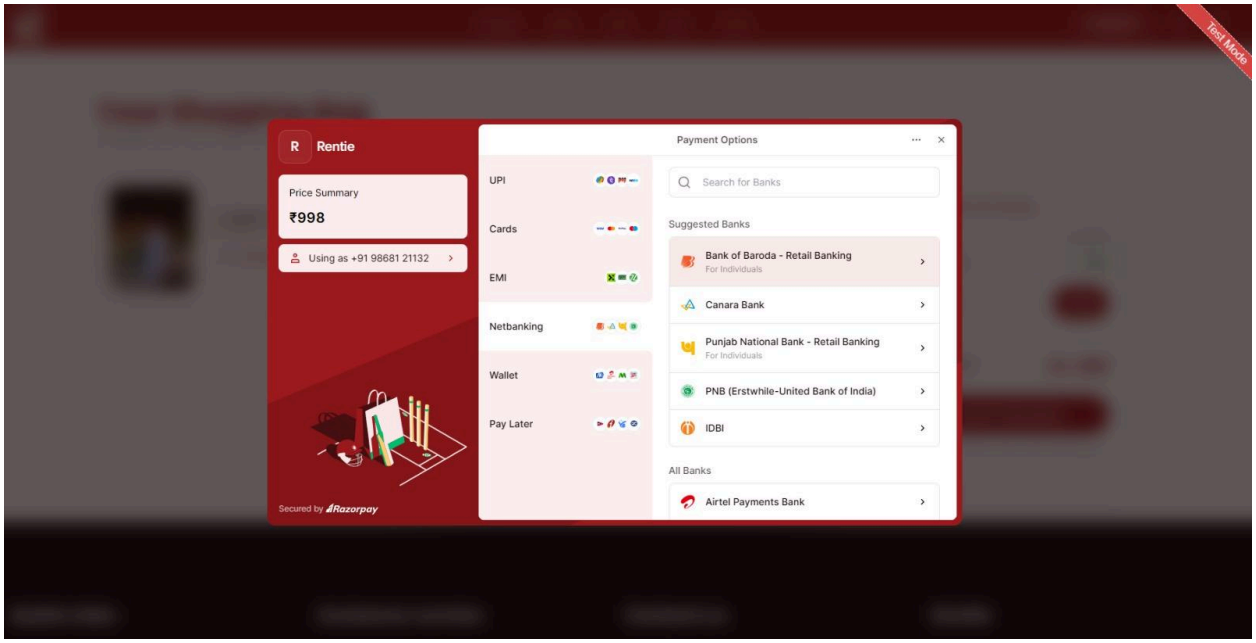
My Cart



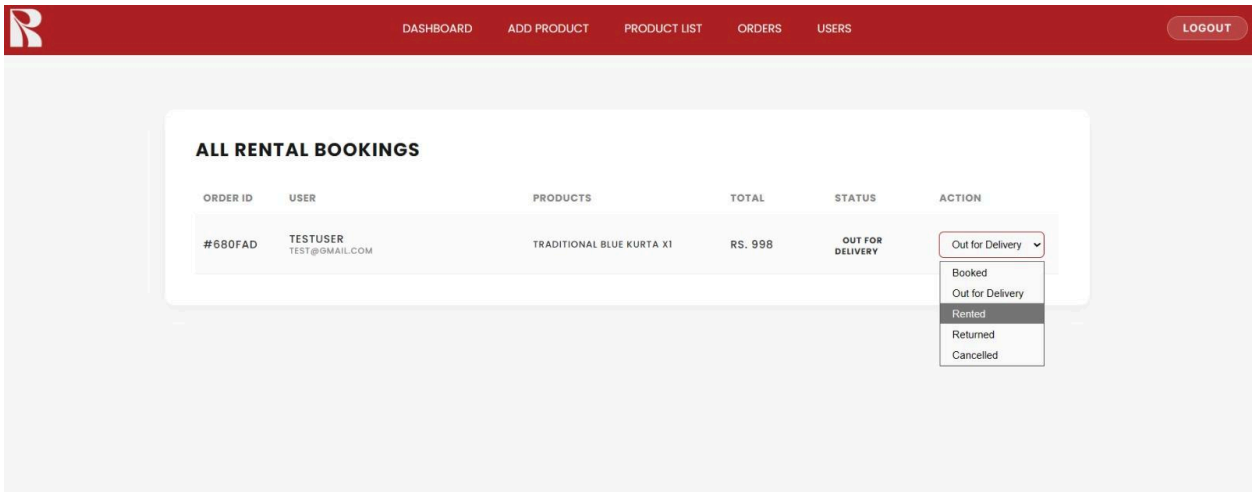
Checkout Page



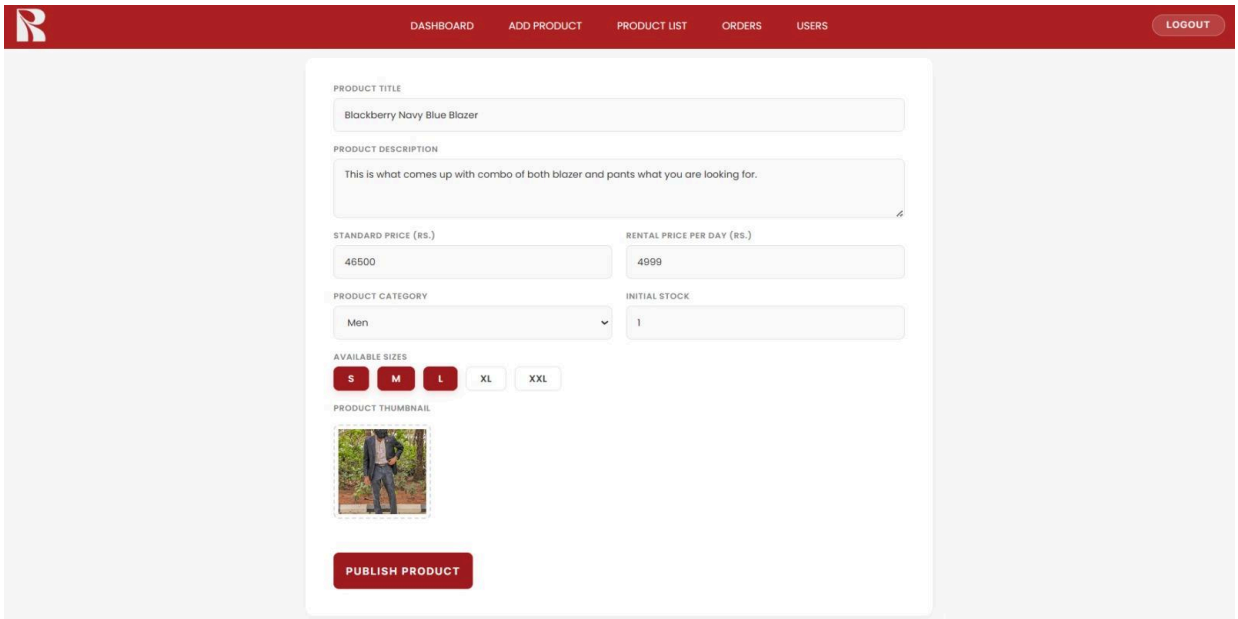
Razor Pay Integration



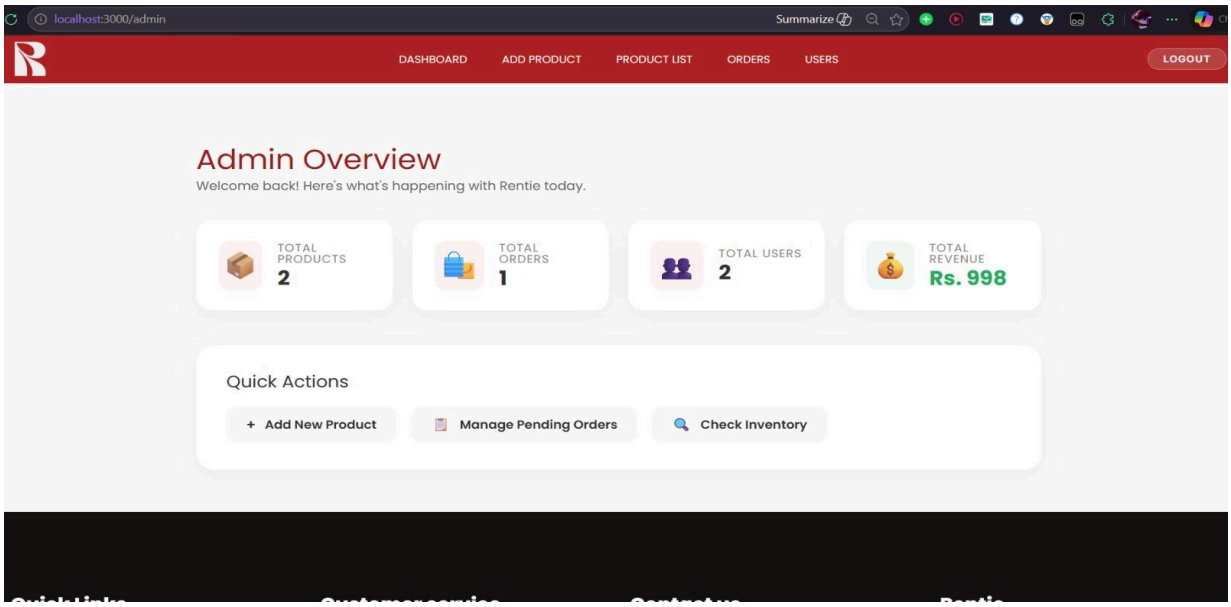
Admin Dashboard/Product List



Add Clothes Page (Admin)



Manage Bookings Page (Admin)



LEARNING OUTCOMES

By completing this project, students will gain comprehensive knowledge and practical experience in:

- **Full Stack Development (MERN):** Build end-to-end applications using MongoDB, Express, React, and Node.js
- **REST API Design:** Design and implement RESTful APIs with proper HTTP methods and status codes
- **Authentication using JWT:** Implement secure user authentication with JSON Web Tokens
- **Database Schema Design (MongoDB):** Design and implement NoSQL database schemas with Mongoose
- **Real-world Booking System Logic:** Handle complex business logic for rental bookings, availability management, and date validations
- **Handling Dates & Availability:** Work with date ranges, prevent double bookings, and manage item availability
- **File Upload (Images):** Implement image upload functionality using Cloudinary or AWS S3
- **Role-based Access Control (Admin/User):** Implement different permission levels for users and administrators
- **State Management in React:** Manage application state using hooks and context API
- **Middleware Development:** Create and use middleware for authentication, validation, and error handling
- **MVC Architecture:** Understand and implement the Model-View-Controller pattern
- **API Testing with Postman:** Test backend APIs thoroughly before frontend integration
- **Deployment Skills:** Deploy full-stack applications to cloud platforms (Vercel, Render, AWS)

CONCLUSION

Rentie is a comprehensive full-stack application that demonstrates modern web development practices using the MERN stack. This project provides hands-on experience in building a real-world rental management system with features like user authentication, inventory management, a secure booking system, and role-based access control, integrated with Razorpay for seamless financial transactions.

By following this documentation and implementing the features described, we have gained valuable experience in full-stack development, database design, API security, and building scalable web applications. The project serves as an excellent portfolio piece and demonstrates proficiency in the MERN technology stack. Ultimately, Rentie aligns with the global shift towards a circular economy, proving that technology can effectively drive sustainable fashion by making premium clothing accessible through a smart, digital-first rental model.